PROJECT FILE OF AI AND ML PROJECT

NAME: MEHER BAITULE

REG NO: 25BAI10819

Project Report: Text-Based Emotion Classification Using Machine Learning

1. Introduction

In today's digital world, people communicate extensively through text messages, social media, and online platforms. Understanding the emotional tone behind this text is important for many applications such as customer feedback analysis, chatbots, and recommendation systems.

This project focuses on building a simple sentiment analysis system using machine learning techniques. The system classifies text into three categories: Positive, Negative, and Neutral.

---

2. Objective

The main objectives of this project are:

- To convert textual data into numerical form using feature extraction techniques
- To implement a machine learning model for sentiment classification
- To classify input sentences into Positive, Negative, or Neutral categories
- To evaluate the performance of the model using accuracy

---

3. Technologies Used

- Programming Language: Python
- Libraries Used:
  - Scikit-learn
  - CountVectorizer
  - Multinomial Naive Bayes

---

4. Methodology

4.1 Data Collection

A small dataset of sentences was manually created. Each sentence was labeled based on its sentiment as Positive, Negative, or Neutral.

4.2 Data Preprocessing

Since machine learning models cannot process raw text directly, the text data was converted into numerical form using CountVectorizer. This method transforms text into a matrix of word frequencies.

4.3 Model Training

A Multinomial Naive Bayes algorithm was used to train the model. This algorithm is suitable for text classification problems because it works efficiently with word count data.

4.4 Model Testing

A separate set of test sentences was used to evaluate the model. The predicted labels were compared with the actual labels to check performance.

4.5 Accuracy Evaluation

The accuracy of the model was calculated using the formula:

Accuracy = (Correct Predictions / Total Predictions) × 100

---

5. Implementation Details

The implementation includes the following steps:

1. Importing required libraries

2. Defining training data and labels

3. Converting text into numerical features using CountVectorizer

4. Training the Naive Bayes model

5. Testing the model on new sentences

6. Calculating accuracy

7. Creating an interactive system for user input

---

6. Features of the Project

- Classifies text into three sentiment categories
- Provides probability for each class
- Interactive user input system
- Simple and efficient implementation
- No external dataset required

---

7. Results

The model performs reasonably well on simple sentences. Some example outputs are:

- “I really love this movie” → Positive
- “This food is terrible” → Negative
- “Today is a normal day” → Neutral

---

8. Limitations

- Small dataset reduces accuracy
- Cannot handle complex or sarcastic sentences
- Limited vocabulary understanding
- Not suitable for real-world large-scale applications

---

9. Future Improvements

- Use a larger and real-world dataset
- Apply advanced techniques like TF-IDF
- Implement deep learning models
- Add preprocessing techniques such as stemming and stopword removal
- Develop a web-based interface

RESULT:

```
===== Sentiment Analyzer (with probabilities) =====
File display t' to quit.

Predicted Sentiment: positive
Class probabilities:
  negative: 0.258
  neutral : 0.13
  positive: 0.612

----------------------------------------

Predicted Sentiment: negative
Class probabilities:
  negative: 0.479
  neutral : 0.11
  positive: 0.411

----------------------------------------

Predicted Sentiment: neutral
Class probabilities:
  negative: 0.217
  neutral : 0.655
  positive: 0.129

----------------------------------------
```

10. Conclusion

This project demonstrates a basic implementation of sentiment analysis using machine learning. It shows how text can be processed and classified into different sentiment categories. Although simple, it provides a strong foundation for further improvements in Natural Language Processing.

---

11. References

- Scikit-learn Documentation
- Python Official Documentation
- Online NLP Tutorials